

**TanStack** (trước đây là React Query) không chỉ là một thư viện, mà là một "hệ sinh thái" các công cụ UI không đầu (headless) cực kỳ mạnh mẽ. Tuy nhiên, trong sự kiện **Mini Shai-Hulud** diễn ra vào tháng 5/2026 này, nó đã trở thành mục tiêu chính của một trong những cuộc tấn công chuỗi cung ứng (supply chain attack) tinh vi nhất lịch sử mã nguồn mở.

Dưới đây là thông tin chi tiết về gói TanStack và cách thức "con sâu cát" này tấn công hệ thống của bạn.

---

## 1. TanStack là gì? (The High-Value Target)

TanStack được xây dựng bởi Tanner Linsley, với triết lý "**Headless UI**" — cung cấp logic, quản lý trạng thái và dữ liệu nhưng để phần hiển thị (UI) hoàn toàn cho lập trình viên quyết định.

- **TanStack Query (React Query):** Quản lý server state, caching và synchronization. Đây là "xương sống" của hàng triệu web app hiện đại.
- **TanStack Router:** Hệ thống routing an toàn về kiểu dữ liệu (type-safe) dành cho các ứng dụng SPA phức tạp.
- **TanStack Table:** Xử lý các bảng dữ liệu khổng lồ với tính năng sorting, filtering và grouping cực mạnh.

Vì độ phổ biến khổng lồ (hàng triệu lượt tải mỗi tuần), việc chiếm quyền điều khiển hoặc đầu độc (poisoning) các gói này mang lại cho kẻ tấn công khả năng truy cập vào hàng ngàn doanh nghiệp lớn.

---

## 2. Giải phẫu cuộc tấn công "Mini Shai-Hulud" (Tháng 5/2026)

Cuộc tấn công do nhóm **TeamPCP** thực hiện nhắm vào các gói thuộc @tanstack thông qua một kỹ thuật gọi là **GitHub Shared Object Storage Trick**.

### Cách thức lây nhiễm:

Kẻ tấn công không trực tiếp sửa code trên repo chính của TanStack. Thay vào đó:

1. Chúng tạo một bản **Fork** của repo tanstack/router.
2. Đẩy mã độc vào bản Fork đó.
3. Sử dụng một URL có dạng github:tanstack/router#<commit\_hash> trong file package.json của các dự án khác.
  - *Kỹ thuật:* Do cơ chế lưu trữ của GitHub, URL này trông như đang trở vào repo chính chủ, khiến các kỹ sư (và cả các tool audit) chủ quan.

## Hành vi của mã độc:

Mã độc này được đặt tên là "Mini Shai-Hulud" vì khả năng "đào hang" và "ký sinh" cực sâu:

- **Bun Runtime Smuggling:** Nó sử dụng Bun để thực thi thay vì Node.js nhằm bypass các công cụ giám sát runtime thông thường (EDR).
- **Credential Harvesting:** Quét sạch các file cấu hình nhạy cảm:
  - `~/.aws/credentials`, `~/.kube/config`, `~/.claude.json`.
  - Đặc biệt, nó đọc bộ nhớ `/proc/{pid}/mem` của **GitHub Actions Runner** để lấy các Secret đang hoạt động.
- **Persistence (Cắm rễ):** Nó không chỉ nằm trong project. Nó tự tiêm (inject) các hook vào:
  - `.vscode/tasks.json` để tự chạy mỗi khi mở folder.
  - `.claude/settings.json` để theo dõi các lệnh bạn yêu cầu AI thực hiện.

---

## 3. Tại sao đây là "cơ ác mộng" đối với các Architect?

Đối với một người quản lý hạ tầng như bạn, rủi ro không chỉ dừng lại ở việc lộ mã nguồn, mà là **mất toàn quyền kiểm soát hạ tầng:**

- **Bypass Vault:** Mã độc này chủ động tìm kiếm các token truy cập **HashiCorp Vault**. Nếu nó lấy được Root Token, toàn bộ hệ thống bảo mật của bạn coi như sụp đổ.
- **Self-Propagation (Tự nhân bản):** Sau khi lấy được NPM token của bạn, virus sẽ tự động xuất bản (publish) các phiên bản mới cho chính các thư viện nội bộ mà bạn đang phát triển, biến bạn thành kẻ phát tán virus cho đồng nghiệp.

---

## 4. Hành động cần thiết ngay bây giờ

**Nếu bạn nghi ngờ hệ thống của mình có liên quan đến các gói TanStack trong giai đoạn từ 29/04/2026 đến 12/05/2026:**

1. **Kiểm tra file `package-lock.json`:** Tìm kiếm bất kỳ dependency nào trỏ trực tiếp đến một commit hash của GitHub thay vì một version cụ thể từ NPM Registry.
2. **Quét các file lạ:** \* Tìm kiếm các file tên `router_init.js`, `router_runtime.js` hoặc `setup.mjs` trong các thư mục ẩn.
  - Kiểm tra sự tồn tại của biến môi trường `__DAEMONIZED=1`.
3. **Vô hiệu hóa Token ngay lập tức:**
  - Rotate toàn bộ Access Keys của AWS và Kubernetes.
  - Thu hồi các token của Claude AI và VS Code extensions.
4. **Sử dụng `govulncheck` (cho các dự án Go):** Mặc dù vụ này chủ yếu ở NPM, nhưng nếu bạn có các module Go sử dụng CGO hoặc gọi ra các script JS, hãy chạy `govulncheck` để đảm bảo chuỗi cung ứng của bạn vẫn sạch.

---

**Sự khác biệt về triết lý:** Cuộc tấn công này một lần nữa khẳng định tại sao việc **Explicit Import** của Go (nhập URL đầy đủ và có file go.sum cực kỳ chặt chẽ) lại an toàn hơn mô hình "dependency của dependency" đầy rủi ro của NPM.

**Bạn có muốn tôi viết một script Go để tự động rà soát toàn bộ các file cấu hình trong cụm Kubernetes của bạn để tìm dấu hiệu của các token đã bị "rò rỉ" ra ngoài không?**